33 計算幾何

Part VII Selected Topics

本章定位

幾何演算法必須把數學判定、退化情形及資料結構順序一致處理；本章以方向測試、交線、凸包及最近點對證明局部判斷如何形成正確全域結果。

前置知識

向量、叉積、排序與掃描線。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用 cross product 判斷線段方向與轉向。
- 以 sweep line 找任兩線段相交。
- 分析 convex hull 與 closest pair。
- 33.1 Line-segment properties
- 33.2 Determining whether any pair of segments intersects
- 33.3 Finding the convex hull
- 33.4 Finding the closest pair of points

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.1 Line-segment properties

平面點p=(x,y)，閉線段ab={a+t(b−a)：0≤t≤1}。閉線段的交集包含端點接觸、共線重疊及退化為單點的線段，不只真正穿越。

定義方向判定

$$\operatorname{orient}(a,b,c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

本章採一般向右的x軸、向上的y軸：正為a→b→c左轉（counterclockwise），負為右轉，零為共線。這是有向平行四邊形面積；三角形面積為其絕對值的一半。若改用(b−a)與(c−a)的相反叉積順序，正負判準也必反轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

方向判定要精確，不能任意套 Epsilon

若整數座標絕對值≤B，座標差≤2B，兩乘積之差的絕對值≤$8B^2$。儲存原座標的整數型別未必能儲存乘積；必須先擴寬再相減／相乘，或使用任意精度整數。b位元座標的判定涉及O(b)位元整數的乘法，並非任意輸入都能O(1)。

浮點運算可能因相消把很小的非零方向算成0。固定ε不能對任意尺度保證正確：縮放座標λ倍會把orient乘λ^2，而拓樸的左／右關係不變。將「差不多共線」當作精確等價，還可能破壞排序比較的傳遞性。

本章正確性採精確實數判定模型；可重算例用整數／有理數精確計算。需要浮點實作時，須使用有誤差界的篩選、失敗後提高精度的精確判定，不能用未證的ε取代符號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線段相交的完整判定

令 o_1=orient(a,b,c)、o_2=orient(a,b,d)、o_3=orient(c,d,a)、o_4=orient(c,d,b)。on(a,b,p)必須同時檢查orient=0與兩軸的閉區間：

```
on(a,b,p):
    return orient(a,b,p)==0 and
           min(ax,bx)<=px<=max(ax,bx) and
           min(ay,by)<=py<=max(ay,by)
intersect(a,b,c,d):
    if sign(o1)*sign(o2)<0 and sign(o3)*sign(o4)<0: return true
    return on(a,b,c) or on(a,b,d) or on(c,d,a) or on(c,d,b)
```

比較符號即可，不必直接乘兩個可能很大的orientation值。垂直線段不需斜率，單點線段也由on處理。若已知共線，on可只檢查bounding box；未知共線時不能省掉第一個條件。

為什麼兩邊都跨越才足夠

在沒有零方向的情形，o_1,o_2異號表示c,d在ab所在直線的不同側；o_3,o_4異號表示a,b在cd直線不同側。前者使cd與ab所在直線的唯一交點落在cd內部；後者使同一個交點也落在ab內部，故兩閉線段相交。

反過來，兩線段若在各自內部穿越，每對端點必分居對方直線兩側。若方向為0，交點不是一般穿越：可能是一端落在另線段上，或兩段共線。兩個共線閉區間有交集時，至少一個端點落在另一區間內；四個on測試完整涵蓋。若其中一段是單點，也正是測該點是否在另段上。

只測一對異號不夠：c,d可以跨過ab的延長線，卻在ab端點之外才相交。

同一判定涵蓋穿越、接觸與分離

取a=(0,0)、b=(4,0)：

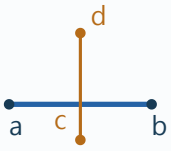
c,d	(o_1,o_2)	(o_3,o_4)	結果
(2,−1),(2,2)	(−4,8)	(6,−6)	內部穿越(2,0)
(4,0),(6,2)	(0,8)	(8,0)	共用端點b
(2,0),(6,0)	(0,0)	(0,0)	重疊[2,4]
(5,0),(6,0)	(0,0)	(0,0)	bounding box分離

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一判定涵蓋穿越、接觸與分離 （續）

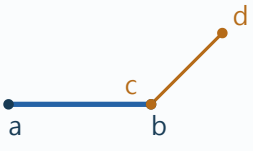
閉線段交集：端點與共線也必判定

內部穿越



$o_1=-4$，$o_2=8$

共用端點



$o_1=0$，$o_2=8$

共線重疊



共同區間 [2,4]

共線分離



bounding boxes 不交

藍段 a=(0,0)→b=(4,0)；四圖 x、y 使用相同單位長度，y 方向向上。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.2 Determining whether any pair of segments intersects

輸入n條閉線段，目標只回傳是否存在一對相交，不要求列出所有交點。所有配對直接判定需$\Theta(n^2)$；若有$\Theta(n^2)$個交點，完整列舉本來就不能有O(n log n)的總輸出界。

先看非垂直線段。每段整理為左端l、右端r，l_x<r_x。在目前掃描位置x，active集合包含l_x≤x≤r_x的線段；status以交點高度y_s(x)由低至高排序。Event（事件）為左右端點。插入新段只測與上下鄰居；刪除前測將成為鄰居的上下兩段。

比較的key不是線段ID，而是當下的幾何高度。對s=(l,r)，

`y_s(x)=[l_y(r_x−l_x)+(r_y−l_y)(x−l_x)]/(r_x−l_x)`。

分母為正，兩段比較可交叉相乘，避免除法。每次精確算術O(1)的模型下，平衡樹每操作O(log n)。

只在端點更新 Status 的演算法

同x事件先處理所有左端，再右端；同類依y、ID確定次序。高度相同用ID作樹內tie-break，但仍對相鄰段做完整閉線段相交測試。

```
T=empty balanced ordered tree
for each endpoint in (x, left-before-right, y, id) order:
    if left endpoint of s:
        insert s in T at this x
        if intersect(s,below(s)) or intersect(s,above(s)): return true
    else:
        a=below(s); b=above(s)
        if both exist and intersect(a,b): return true
        delete s from T using its stored node handle
return false
```

不存在的鄰居略過。交集測試針對整條線段，所以可能在實際到達交點以前就回傳true。此版本一旦找到就停止；不能把return改成print後照舊繼續，因越過交點後status順序會改變。

鄰居測試為何不漏：最早交點證明

若回傳true，必由精確兩段判定找到真交集，沒有誤報。反證存在交集卻全程回false。選最小x的交點p。p之前沒有任何交叉，active段的高度順序不會自行交換，所以status仍正確。

若兩條在p相交的段在p左側之間還有別段，該段的高度被兩者夾住，連續性使它也通過p；在通過p的有限段群中，至少有一對在p之前相鄰。若相交始於端點，則在該x插入時與相同高度群的鄰居相接；先插後刪保留了恰於同點結束的段。

這對段第一次成為鄰居，只可能由插入其中之一，或刪掉它們之間最後一段造成。兩種事件都做完整交集測試，應已回true，矛盾。共線重疊同樣在重疊開始的端點形成相同高度群，其相鄰段也相交。

這也解釋為何不用對每個x重新排序：尚未找到交集前，順序無改變；若即將改變，先前的鄰居檢查已使演算法停止。

刪除中間段時才出現的新鄰居

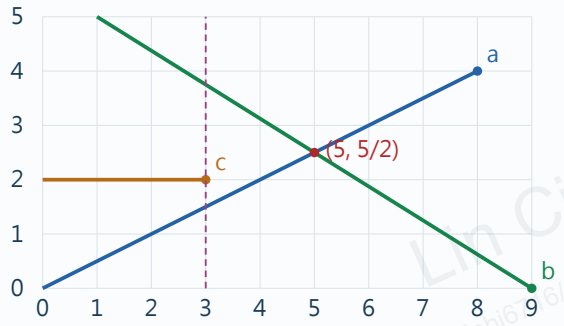
a:(0,0)→(8,4)，b:(1,5)→(9,0)，c:(0,2)→(3,2)。a,b交於(5,5/2)，c與另兩段都不相交。

x與事件	status由低至高	新測試
0插a	a	無
0插c	a,c	a,c不交
1插b	a,c,b	c,b不交
3刪c之前	a,c,b	a,b相交，回true

在x=3三段高度分別3/2、2、15/4，刪c後a,b才相鄰。只測新插入線段會漏掉這類情況。

刪除中間段時才出現的新鄰居 (續)

刪除 c 後，才需要檢查新鄰居 a、b



x=3 的高度：

a：3/2

c：2（將刪除）

b：15/4

刪除前先測 a,b

交點在未來也立即接受

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

垂直線段、端點與成本的完整契約

對一般閉線段，按相同x分批處理：先插入該x開始的非垂直段；再處理該x所有垂直段；最後刪除該x結束的非垂直段。如此端點接觸仍包含於active集合。

垂直段v的y範圍為[L,H]。在status找高度≥L的第一段，若其高度≤H就相交；否則沒有非垂直段在範圍內。這是有序樹lower_bound，O(log n)，不是把整段v硬放成只有一個高度的key。同x的垂直段依L排序；維護已見最大H，若下一個L≤最大H即有重疊，否則更新最大H。單點線段也放入此閉區間群。

排序O(n log n)，2n次非垂直插／刪及n次以內range query各O(log n)，垂直群排序的總成本不超過O(n log n)，故一般版本O(n log n)、空間O(n)。這個界採精確常數成本幾何判定；b位元整數座標的交叉乘積成本須另乘相應位元成本，不能省略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.3 Finding the convex hull

凸集包含其任意兩點間整條線段。有限點集Q的凸包CH(Q)是所有凸組合Σλ_ip_i（λ_i≥0、Σλ_i=1）的集合，也是包含Q的最小凸集。

輸出約定：重複座標先去重，只輸出極端頂點，逆時針排列，末尾不重複首點；位於同一條邊中間的點不另列。空集輸出空；單點輸出該點；全部共線且至少兩點時只輸出兩個端點。

Graham scan按極角排序後維護候選stack；Jarvis march每次選下一條支持邊，成本依凸包頂點數h。兩者不需要以浮點atan2計角度：精確orientation與距離平方比較足以決定順序及共線取捨。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Graham Scan：先處理角度相同的點

取p_0為y最小、同y時x最小的點，其餘點都在上半平面或正x方向。對u,v以orient(p_0,u,v)>0判u極角較小；零時比較離p_0距離平方。相同射線只保留最遠點，其餘在p_0和最遠點的線段內，刪掉不改凸包。

```
S=[]
for p in [p0]+points_in_strictly_increasing_polar_angle:
    while len(S)>=2 and orient(S[-2],S[-1],p)<=0:
        pop S
    push p
return S
```

長度檢查使不足三點時也不讀空stack。全部共線先回端點；非共線時掃描結果閉合為凸多邊形。用≤0刪去平角中間點，與本章「只輸出極端頂點」的契約一致；若要求列出每個邊界點，必須另改同射線及最後共線鏈的處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pop 的幾何理由：舊頂點已被新三角形包含

固定一次新點p_i。Stack由p_0向上按極角遞增，考慮倒數兩點r,t。若orient(r,t,p_i)≤0，則t不是新凸包頂點。

若$r=p_0$，因t與p_i的極角嚴格遞增且跨度小於π，orient(p_0,t,p_i)>0，這一輪不會pop。因此真正需要pop時$r\neq p_0$。把p_0平移到原點，r,t,p_i的射線按角度依序排列且跨度小於π，所以t位於r與p_i張成的錐內，可寫$t=\alpha r+\beta p_i$，$\alpha,\beta\geq 0$。令D=orient(0,r,p_i)>0，則

`orient(r,t,p_i)=(α+β−1)D`。

非左轉推出$\alpha+\beta\leq 1$，所以$t=\alpha r+\beta p_i+(1-\alpha-\beta)p_0$，是三角形$p_0$,r,p_i的凸組合。刪t不改變加入p_i後的凸包；等號恰為在r,p_i邊上，仍非極端頂點。這是可以反覆pop的證據，不是只以「凸包不能右轉」當充分理由。

Stack 不變量與完整正確性

處理p_i前，stack恰為已處理點凸包的逆時針頂點。初始一／兩點符合退化凸包；首次非共線點形成三角形。

新p_i的極角比先前點大，$p_0\to p_i$是新集合的一條支持邊：所有已處理點都在此有向邊的右側或線上。從舊凸包末端向前看，對p_i不可見的舊邊不變；可見的末端連續鏈被新邊取代。每個pop按上一頁證明是新三角形內的非極端點，不改CH；直到末端(r,t,p_i)為左轉，$t\to p_i$成為另一條切線，留下的舊凸邊與$p_i\to p_0$圍成凸多邊形。

此多邊形包含舊凸包及新點，且頂點全來自輸入，故同時包含CH且包含於CH，兩者相等。歸納到最後即得到CH(Q)。每點push一次、pop至多一次，掃描O(n)；初始排序O(n log n)主導，空間O(n)。

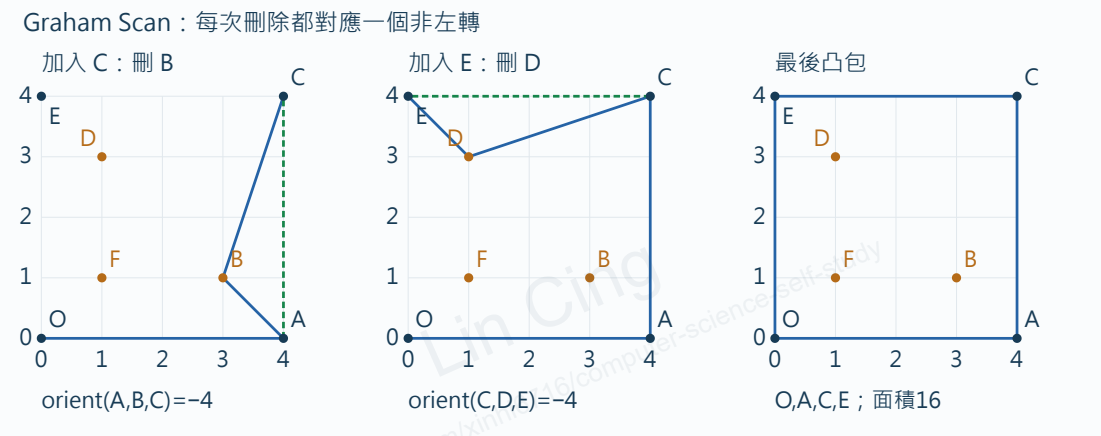
凸包的兩次 Pop 都能用數值核對

p_0=O(0,0)、A(4,0)、B(3,1)、C(4,4)、D(1,3)、E(0,4)，極角次序O,A,B,C,D,E。另有F(1,1)，與C同射線，先刪F。

新點	判定	Pop後／加入後stack
O,A,B	orient(O,A,B)=4	O,A,B
C	orient(A,B,C)=−4，刪B；orient(O,A,C)=16	O,A,C
D	orient(A,C,D)=12	O,A,C,D
E	orient(C,D,E)=−4，刪D；orient(A,C,E)=16	O,A,C,E

B=O/4+A/2+C/4，D=O/4+C/4+E/2，正是被其他點的凸組合包含。最後正方形頂點O,A,C,E，面積16。

凸包的兩次 Pop 都能用數值核對 (續)



Jarvis March：每步找到一條支持邊

從字典序最小點p開始。選任一q≠p，掃描所有r≠p：若orient(p,q,r)<0就令q=r；共線且r更遠也令q=r。掃完後所有點都在p→q左側或線上，q就是逆時針下一個極端頂點。輸出p，再令p=q，回到起點時停止。

為何一次掃描足夠？p是凸包頂點，所有其餘點方向落在一個跨度至多π的支持半平面；在這個半平面內，orientation決定方向的線性次序。每次更新保留已掃點中最靠順時針的射線；同射線保留最遠端。掃完便是一條支持邊，其他點位於同一側。從極端點走到下一極端點，直到繞回，不會走入內點。

每步檢查n−1點，共h條邊，時間O(nh)，空間O(h)含輸出；h≤n，所以最壞O(n^2)。同一正方形例每步選A,C,E,O，h=4。它在h很小時有利，但不能無條件稱比Graham快。

33.4 Finding the closest pair of points

給n≥2個帶唯一ID的點，求歐氏距離最小的一對。重複座標允許，答案可為0；若n<2回報「無點對」。比較距離平方即可避免為每個候選求平方根。

先依(x,y,ID)排X、依(y,x,ID)排Y。每次按X的前⌈n/2⌉個ID分成L，其餘R；分割線ℓ選在兩組x範圍之間（同x可取該x）。沿Y一次穩定分流成Y_L,Y_R，必用ID membership，不能只用x<ℓ，否則所有點同x時會分不平衡。

遞迴求兩側最佳距離δ_L、δ_R，令$\delta=\min$。若$\delta=0$可直接回傳。任何更近的跨組點對，兩點距分割線都<δ，且縱向距離也<δ，所以只需檢查寬2δ的strip內候選。

最近點對的分治偽碼

```
closest(X,Y):
    if len(X)<=3: return brute-force closest pair
    split X by rank into XL,XR; select dividing x=ell
    split Y by IDs into YL,YR, preserving y order
    best=min(closest(XL,YL), closest(XR,YR))
    delta=distance(best)
    if delta==0: return best
    Z=[p in Y with (px-ell)^2 < delta^2]
    for i=0..len(Z)-1:
        for j=i+1..min(i+7,len(Z)-1):
            if (Z[j].y-Z[i].y)^2 >= distance(best)^2: break
            if distance2(Z[i],Z[j]) < distance2(best): best=(Z[i],Z[j])
    return best
```

Strip範圍使用兩側答案的固定δ；比較時best可變小，但每點最多看後7點。只求一對時，與已知最小值相等的候選不必再更新，因此可以使用嚴格strip與≥break。若要求輸出所有最小點對，需要另設相等距離與輸出規則。

為何只需後七點：分組的裝箱證明

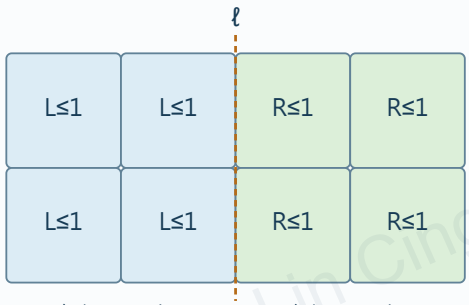
固定strip內一點p，只可能改善答案的後續點q滿足0≤q_y−p_y<δ，故落在寬2δ、高δ的矩形。左組點位於左半δ×δ，右組點位於右半δ×δ；同一組任兩點距離至少δ，這是遞迴最小值的保證。

將每個半正方形分為四個δ/2×δ/2小格，以固定半開邊界分配每點。單格直徑δ/√2<δ，所以同組單格至多一點，每組至多4點，合計至多8點。扣除p本身，其餘至多7點，q在依y排序的後7個位置內。

落在分割線上的點按ID屬於其中一組，各用自己的半正方形計帳；不能把兩組都在分割線的點誤當成同組而排除。若跨組有重複座標，距離0也會由此候選比較找到；若同組已重複，遞迴早已回0。

為何只需後七點：分組的裝箱證明 （續）

寬 2δ、高 δ：依左右組別，各自最多四點



同組任兩點距離 ≥δ

小格邊長 δ/2

小格直徑 δ/√2<δ

含 p 共至多8點

所以只看後7點

分割線上的點仍按 ID 歸組；同一幾何位置可各有一個 L、R 點。

最近點對的完整合併數例

X依序為A(0,0),B(1,4),C(2,1),D(3,7),E(4,1),F(5,5),G(6,0),H(7,8)。左組A–D，右組E–H，ℓ=7/2。

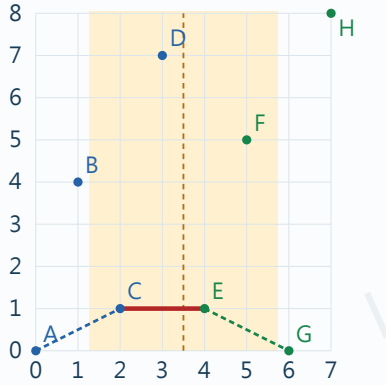
左組最佳A,C，距離平方5；右組最佳E,G，也為5。δ=√5，strip包含C,E,F,D，按y順序正是C(2,1)、E(4,1)、F(5,5)、D(3,7)。

目前點	後續檢查	結果
C	E：平方距離4；下一F的y差4	best改C,E；4≥2，停止此列
E	F的y差4	不可能改善2，停止
F	D的y差2	相等下界也不會改善，停止
D	無後點	結束

跨組C,E的距離2改善兩個子問題的√5。圖中strip按固定√5畫，不隨best縮窄。

最近點對的完整合併數例 （續）

兩側答案 √5，跨組 C、E 的距離 2 更小



$\ell = 7/2$，固定 $\delta = \sqrt{5}$

左組 A–D，右組 E–H

Strip 依 y 排序：C,E,F,D

$|C-E|^2 = (4-2)^2 + 0^2 = 4$

兩側最短距離平方都是5

檢查跨組後 $best^2 = 4$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

複雜度來自穩定分流，不是每層重排

每次分割X用rank，Y以ID穩定分流，建立strip與每點至多7次比較，非遞迴工作O(n)。故

$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) = O(n \log n)$。

最初兩次排序也O(n log n)。若每個子問題重排Y，合併工作變O(n log n)，總界便$O(n \log^2 n)$。ID membership可預先記每點在本層的左右標記；穩定分流不能對每個Y點線性搜尋整個L。

按深度先遞迴、子問題返回後釋放暫存，單條遞迴鏈的陣列總長$n+n/2+\cdots=O(n)$，加當前兄弟與回傳資訊仍O(n)工作空間，stack O(log n)。若刻意保存整棵遞迴樹的所有陣列，會用O(n log n)，那不是這個線性空間實作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 33。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study